MODULE 03 / TEAMMATE FIELD GUIDE

Profile Builder

Combine reviewed resume information with manual details into one saved candidate profile. This is the stable record that both assessment modules use.

JD | Resume | **Profile** | Talent Check | Match + GPS

The shared journey; each module also runs independently.

You receive	You return
Manual details, an optional reviewed resume ID, optional CV upload, and twelve optional 1-10 self-ratings.	A saved Profile with an ID/revision, background fields, source-linked claims, CV reference and exactly twelve rating entries. Unknown ratings remain null.

Your responsibility

Participant Brief p.8 requires basics, skills, hackathons/internships, certifications, preferred roles and CV upload, with save/load. Resume prefill is desirable. Evidence passport and explicit self-ratings support our integrated design.

A successful handoff looks like this

Create a manual candidate, save/reload it, import a reviewed resume without losing manual work, and show the exact same saved profile in both assessment modules.

> **Start from the working baseline**
>
> Assume YOU own this module. Run and understand the supplied working source, finish or improve your part, and test it. Push the verified working copy to YOUR OWN GitHub repository. Return its URL, commit SHA, actual test results and known limitations to the Scrum Master. No central merge or PR is required from you.

Before your module	After your module
Module 2 hands you a reviewed resume ID. Your form must also work independently with manual input; it cannot require parsing first.	Modules 4 and 5 receive the same saved profile ID/revision. They must not use a second, differently shaped candidate object.

Read in order: 2 build plan | 3 screen layout | 4 API | 5 logic | 6 integration | 7 tests | 8 presentation | 9 exact field reference

02 / IMPLEMENTATION PLAN

What to do, in order.

Keep changes inside your feature folder. Each step has an observable result; do not begin by replacing the whole app.

01 Start manually

Create a Profile with a nonblank name. Let defaults fill arrays and twelve null ratings. Do not block consumers waiting for resume extraction.

02 Build all sections

Keep name/email, education, projects, experience, internships, hackathons, certifications, preferred roles and achievements. Treat multiline entries as string arrays.

03 Import a reviewed resume

POST import-resume with the current profile draft. Default replace_existing=false preserves nonempty manual fields; import returns a preview, not a saved record.

04 Preserve evidence

Keep source_document_id, exact quotes, confidence, claim basis and review state on imported skills. A duplicate name must not hide a new conflicting claim.

05 Separate ratings

Show twelve Unknown/1-10 controls. Parse Unknown as null, not zero. Keep self_reported origin. Importing evidence cannot increase any rating.

06 Save safely

Use POST for create and PATCH with the full current Profile/revision for updates. Retain drafts on failure; reconcile 409 conflicts rather than silently overwriting.

07 Prove both consumers

Reload the saved record and select it in Talent Check and Skill Matching. Export a small synthetic handoff only; do not share private CVs or keys.

Files to open first

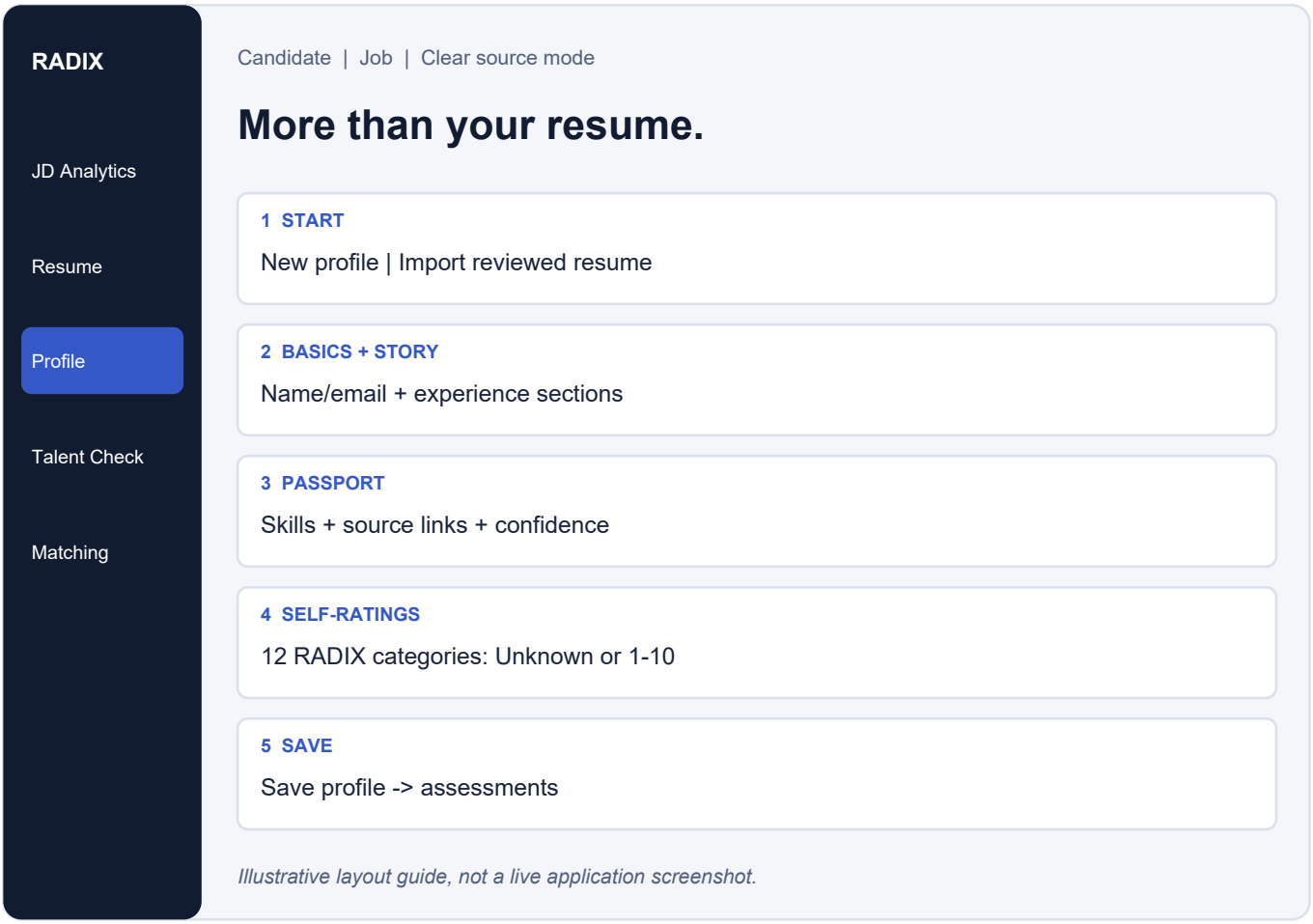
All paths below are relative to profile_builder/.

File	What it owns
backend/service.py	Reviewed resume merge and source validation.
backend/router.py	Profile create/load/update/delete and import-preview routes.
frontend/ProfileBuilderPage.tsx	Full form, CV attachment, evidence passport and self-ratings.
tests/test_profiles.py + test_final_merge.py	Persistence, revisions, merge, sources and conflicting claims.

03 / SCREEN AND USER EXPERIENCE

What it should look like.

Keep the current app shell, navigation, typography and shared controls. The drawing shows information order, not a redesign request.



Screen state	Expected visible behavior
New draft	All twelve rating values unknown; no automatic score or evidence.
Imported preview	Fields are editable; tell the user to save before assessment.
Saved / reloaded	Same ID, current revision and saved CV/claims remain visible.
Save fails / conflict	Keep draft and show retry/reload guidance; no false success.

Visual rules: 16 px body/form text, 14 px minimum supporting labels; clear headings; 44 px touch targets. Navy #111C30, blue #3459C6, white cards and #F4F6FA canvas. Use the existing Inter web font. On mobile, stack panels and keep actions readable. Do not rely on color alone for status.

04 / EXACT API BOUNDARY

Input, output and format.

Use the current Pydantic/OpenAPI contract. A UI label can change; an agreed JSON field cannot be renamed independently.

Endpoint	Body or result
POST /api/profiles	Create validated Profile; returns 201 and saved revision.
POST /api/profiles/import-resume	resume_id + profile + replace_existing=false; merged draft only.
GET /api/profiles[/ID]	List/load owned profiles.
PATCH /api/profiles/ID	Full Profile with current revision; increments on success.
DELETE /api/profiles/ID	Delete owned profile and invalidate receipts.
POST /api/documents	Multipart CV attachment; use returned document ID as cv_file.

Minimal complete create request; defaults are server-filled

```
{
  "name": "Synthetic Candidate",
  "email": "candidate@example.test"
}
```

Current synthetic response excerpt

Only selected fields are shown below. Page 9 describes the complete shape. Snapshot UUIDs are teaching examples, not IDs to reuse in a running app.

```
{
  "id": "b3a2dddc-7e63-5437-a1ef-f85a9f887f4c",
  "revision": 1,
  "name": "Mira Sen (synthetic)",
  "email": "mira@example.test",
  "cv_file": "cbe3412c-d457-56de-98bd-67f54cb8636e",
  "preferred_roles": [
    "Junior Data Engineer"
  ]
}
```

Every mutation needs X-ProofPath-Client: web (or cli). Cloud requests also need the current user's Bearer JWT. The shared web helper supplies these headers. File upload uses multipart; never manually set its boundary. JSON fields use snake_case.

05 / REASONING AND BUSINESS RULES

Import fills a draft; only Save makes it authoritative.

Module 3 owns candidate persistence. A parsed resume is an input, not a replacement for the user's existing profile. Keep review provenance and competency separate all the way through.

Situation	Exact expected reasoning
Import with replace_existing=false	Fill empty background fields from the reviewed resume. Keep nonempty manual values. Merge skill names without losing a new conflicting source.
Import with replace_existing=true	Explicitly replace imported background fields. Understand that current skill merging is still conservative; this option is not a blanket overwrite of all skill claims or ratings.
User adds evidence	Add the source-backed claim if valid. Do not change a category rating, confidence-based proficiency or assessment score automatically.
Save profile	Validate source ownership/quotes. Reject hypothetical source claims. Persist the current revision, and invalidate old evaluation receipts on changes.
User selects Unknown	Store null. Numeric values must be finite 1-10, not booleans or 0. Keep exactly one row for each official category.

Important edge case

The Living Skill Passport currently persists the current claims, source links and profile revision. There is no historical timeline or immutable evidence archive UI. Present what exists; do not promise an implemented history view.

```
POST /api/profiles/import-resume
{
  "resume_id": "CURRENT_REVIEWED_RESUME_ID",
  "profile": {"name": "Synthetic Candidate"},
  "replace_existing": false
}
// Response is a merged draft: save it explicitly.
```

Never cross this boundary

A manually entered skill is self-reported, not document-verified. All verification_status values remain unverified. Source deletion removes dependent document claims; saved self-ratings remain self-reported.

06 / INTEGRATION AGREEMENT

Own it. Test it. Publish your part.

Your task is a working module on your own GitHub. Keep the supplied shared contracts and exact output formats so the Scrum Master can integrate the returned code.

Team rule	What you do
Service boundary	backend/service.py: import_resume(request, resume) -> Profile preview; validate_sources(profile, user) before save. The router owns HTTP/storage calls.
One feature owner	Own profile_builder/backend, frontend, tests and docs. Keep the common support area in your clone. Run your module independently before claiming it works.
One contract source	Read shared/contracts/models.py. OpenAPI and generated TypeScript follow it. Do not maintain a second handwritten schema or change categories alone.
One storage/auth path	Use shared store/document/auth helpers. Do not add a second database, anonymous cloud owner, public CV URL or frontend API secret.
One current record	Use actual returned IDs/revisions, owned by the signed-in user. Pass saved IDs to consumers; never hardcode teaching snapshot IDs.
Your GitHub	Create a private repository on YOUR account. Rename the old origin to scrum-master, set your new origin, review staged files and scan for secrets, then commit and push. See SUBMIT_YOUR_REPOSITORY.md.
Return working evidence	Fill DELIVERY.md with your URL, commit SHA, actual commands/results, three normal and three failure cases, synthetic screenshots and known bugs. If a check fails, fix it or report it honestly; do not copy passing counts.

The official category vocabulary

Every extracted skill has exactly one official code or OTHER. The extra descriptive skill families are filters, not replacement categories. Only the twelve official codes can have numeric ratings or company expectations.

Category	Category
COD - Coding	CLOUD - Cloud
DSA - Data Structures & Algorithms	SQL - SQL
OOD - Object-Oriented Design	SWE - Software Engineering
APTI - Aptitude	SYSD - System Design
COMM - Communication	NETW - Networking
AI - AI	OS - Operating Systems

Keep these meanings separate

Confidence = how clearly the document supports an extraction. Competency = explicit self-rating, null or 1-10. Review = a human checked the output. Verified = independently established; this app's claims remain unverified. Hypothetical = GPS preview only.

07 / RUN AND VERIFY

Your definition of done.

A module must run independently as well as inside the shared app. Record actual commands and failures; a screenshot or a copied passing count is not test evidence.

Use the existing project, not a second app

From the repository root, install the existing requirements/lockfiles if needed. Open two terminals. If the ports are already running, use them; do not kill unknown processes. Never overwrite an existing .env.

```
.\profile_builder\run.ps1 api
# Separate terminal:
.\profile_builder\run.ps1 web
# Real standalone API check:
.\profile_builder\run.ps1 check
```

Clone the full repo and install root dependencies (README). Run these from its root. Open http://127.0.0.1:5173/; Load synthetic inputs supplies upstream fixtures. No key is needed for rules. A Groq key belongs only in the private root .env; restart Python after editing it.

Test	Pass means
Manual profile without parsing	Create/save/load works; all twelve ratings default to null.
Reviewed vs unreviewed resume import	Unreviewed input rejected; reviewed output merges into a draft and requires Save.
Manual field + duplicate/conflicting claim	Manual work is preserved; new contradictory evidence is not silently discarded.
Invalid rating/email/source	Reject invalid values, missing/unowned documents and hypothetical claims.
Save/reload and stale revision	Persistence works; stale PATCH returns 409 without losing edits.
Both assessment consumers	Same saved ID/revision used in Modules 4/5; scenario preview never mutates it.

Your focused commands

```
.\.venv\Scripts\python.exe -m pytest profile_builder/tests -q
npm run test -- profile_builder/frontend
```

Before returning: run typecheck, lint, build and the standalone browser check. Use the project virtualenv for Python. Keep assertions meaningful and record actual results.

Error behavior to demonstrate

422 = invalid/unreadable input; 413 = size limit; 415 = unsupported file; 404 = missing/unowned record; 409 = stale revision; 503 = unavailable storage/benchmark. Groq may fall back and return a successful Extraction with llm_status=unavailable instead of an HTTP error. Show the status, not just HTTP 200.

08 / LIVE DEMO AND PRESENTATION

Show it. Explain it. Hand it over.

Suggested speaking time: four minutes per module. Five modules use about 20 minutes, leaving 10 for the problem, architecture, challenge and close within the 30-minute presentation. Q&A is separate (15 minutes).

Reproducible synthetic demo

1. Open Profile Builder. Import the reviewed Mira resume. Explain that this creates an editable draft.

2. Show education/projects/internship, CV link and the six source-linked skills. Add preferred role Junior Data Engineer if desired.

3. Point out all twelve ratings are initially Unknown. This is the important invariant after parsing/import.

4. For the synthetic numeric demo only, deliberately set every category to 4 and say these are invented self-ratings. Click Save profile.

5. Reload the page, show the saved version and unchanged fields, then hand the same profile ID/revision to Modules 4 and 5.

What to say and show

Time	Speaking cue
0:00-0:35	My module turns extracted evidence and manual information into one saved candidate record.
0:35-1:25	Show the reviewed resume import and additional story fields. Explain that import previews, Save persists.
1:25-2:15	Open a source claim and show Unknown ratings. Explain why evidence confidence is not proficiency.
2:15-3:10	Enter clearly labelled synthetic ratings, save and reload. Describe revision checks and private ownership.
3:10-4:00	Hand the same profile ID to both assessors. Explain one conflict/failure behavior and the passport's current limits.

Transition to the next speaker

The profile is now saved. Module 4 compares its self-ratings with a selected company's benchmark, while Module 5 compares named claims with one JD.

Use this language honestly

Say which parts you implemented, adapted, tested or learned. If showing the working team baseline, call it the integrated team implementation. Do not claim a live model, official benchmark or feature history that has not actually been demonstrated.

09 / FIELD REFERENCE AND QUESTIONS

Keep this beside your editor.

These are exact field names from the current Python contracts. Defaults fill omitted optional fields; partial excerpts are not safe replacements for full PATCH payloads.

Group	Exact fields / meaning
Profile scalar fields	schema_version, id, revision, name, email, cv_file
Background string arrays	education, hackathons, internships, certifications, preferred_roles, projects, experience, achievements
Skills array	Skill objects: identity/category/family, exact source evidence/provenance, confidence, claim basis and review state. Keep all fields from the shared model.
Ratings array	Exactly twelve entries: category_code, level=null|1-10, origin=self_reported. OTHER cannot be rated.
Import request	resume_id, profile (the full current draft), replace_existing=false. Returns a Profile preview without saving.
CV reference	cv_file is an owned document ID, not a filename or public download URL.

Questions you should answer without memorizing code

Why not rate from the resume automatically? Mentions and confidence are not measured ability. Unknown stays null; the user supplies explicit self-ratings.

What happens when two people edit? Current revision checks reject stale saves with 409. The UI keeps the draft for reconciliation.

Is it a full evidence history? It saves current claims, sources and revision. A historical timeline is not implemented.

Source of truth and current limits

Read your module's README.md, PLAN.md, CONTRACT.md and USAGE.md; shared/contracts/models.py; docs/BUILD_PLAN.md; docs/DEPLOYMENT_SETUP.md; docs/LLM_EXTRACTION_IMPLEMENTATION.md; verification/RESULTS.md; and docs/TEAM_PRESENTATION_CHECKLIST.md. Full current validated synthetic JSON snapshots are under data/synthetic/handoff/. Snapshot IDs are not live IDs.

Original Participant Brief: pp.7-10 (roles/contracts), p.11 (official test pack), p.13 (stack/done). Event Plan p.4: 30 minutes presentation + 15 Q&A. Original baseline suite (see new checkout results in verification/RESULTS.md): 134 backend, 17 frontend and 14 browser tests. Live Groq quality, cloud RLS/deployment and official organizer-pack/benchmark acceptance remain pending. All guide examples are synthetic.